Days 11-14 — SQL and the EDA portfolio project: complete study guide

MD Mutasim Billah · 52-week Data Science to ML/AI roadmap · Week 2 close-out — definitions, real-world examples, implementations, and an interview-style quiz

Same format as the earlier guides: definition, real-world analogy, implementation, then a quiz. Days 11-14 were taught directly in this roadmap, so this content matches your actual scripts (day11_sql_basics.py, day12_sql_intermediate.py, day13_eda_portfolio.py) exactly.

Day 11 — SQL basics

Definition

> **Definition — SQL (Structured Query Language):** the standard language for querying relational (table-based) databases — SELECT, WHERE, GROUP BY, and JOIN are the core vocabulary for retrieving, filtering, aggregating, and combining data stored across tables.

> **Definition — WHERE vs HAVING:** WHERE filters individual rows before any grouping happens; HAVING filters groups after GROUP BY has already aggregated them. You can't use HAVING without GROUP BY, and you can't filter on an aggregate function (like AVG()) inside WHERE.

Real-world example

Real-world example: "Show me every passenger over 60" is a WHERE clause. "Show me only the passenger classes where the average fare was above 50" is a HAVING clause, because 'average fare' doesn't exist until after grouping by class. This exact distinction is one of the most commonly tested SQL concepts in interviews because it trips up so many beginners.

Implementation

```
import sqlite3
import pandas as pd

conn = sqlite3.connect("titanic.db")
df.to_sql("passengers", conn, if_exists="replace", index=False)

query = """
    SELECT Pclass, AVG(Fare) as avg_fare, COUNT(*) as n
    FROM passengers
    GROUP BY Pclass
    HAVING AVG(Fare) > 50
    ORDER BY avg_fare DESC
"""
result = pd.read_sql(query, conn)

join_query = """
    SELECT p.Name, p.Pclass, c.ClassLabel
    FROM passengers p
    INNER JOIN class_lookup c ON p.Pclass = c.Pclass
"""
```

- `pd.read_sql(query, conn)` — runs a SQL query against the SQLite connection and returns the result directly as a pandas DataFrame, the bridge between SQL and pandas used throughout this roadmap
- `INNER JOIN` only keeps rows that match in both tables; `LEFT JOIN` keeps every row from the left table even without a match, filling unmatched columns with NULL
- GROUP BY, HAVING, and ORDER BY have a fixed required order in a SQL statement — writing HAVING before GROUP BY is a syntax error, not just a style issue

Day 12 — SQL intermediate

Definition

Definition — Subquery: a query nested inside another query, either producing a single value used in a WHERE comparison (scalar subquery) or a list of values used with IN.

Definition — CTE (Common Table Expression): a named, temporary result set defined with WITH ... AS (...) that can be referenced later in the same query, making complex multi-step logic far more readable than deeply nested subqueries.

Definition — Window function: computes a value across a set of rows related to the current row (like RANK() or ROW_NUMBER()) without collapsing those rows into a single aggregated result the way GROUP BY does — every original row is preserved, with a new computed column added.

Real-world example

Real-world example: "Show me the top 3 highest fares within each passenger class" cannot be done with a plain GROUP BY, because you need to keep all the individual rows, not collapse them. That's exactly what RANK() OVER (PARTITION BY Pclass ORDER BY Fare DESC) is for — it ranks rows within each class group while keeping every row visible.

Implementation

```
cte_query = """
    WITH class_avg AS (
        SELECT Pclass, AVG(Fare) as avg_fare
        FROM passengers
        GROUP BY Pclass
    )
    SELECT p.Name, p.Pclass, p.Fare, c.avg_fare
    FROM passengers p
    JOIN class_avg c ON p.Pclass = c.Pclass
    WHERE p.Fare > c.avg_fare
"""

window_query = """
    SELECT Name, Pclass, Fare,
           RANK() OVER (PARTITION BY Pclass ORDER BY Fare DESC) as fare_rank
    FROM passengers
"""
top_fares = pd.read_sql(window_query, conn)
top_3_per_class = top_fares[top_fares["fare_rank"] <= 3]
```

- `WITH class_avg AS (...)` — defines class_avg as if it were a temporary table, readable top to bottom, versus writing the same subquery nested inline inside the FROM clause
- `PARTITION BY Pclass` — resets the ranking separately within each Pclass group, the window-function equivalent of GROUP BY, but without collapsing rows
- `RANK()` leaves gaps after ties (1, 2, 2, 4); `ROW_NUMBER()` never ties, assigning a unique sequential number even to identical values — a common interview distinction

Day 13 — Full EDA portfolio project

Definition

Definition — EDA (Exploratory Data Analysis): the systematic process of understanding a dataset before modeling it — cleaning, summarizing, visualizing, and documenting findings, so that any modeling decisions later are grounded in an accurate understanding of the data.

Definition — Clean-once pipeline design: cleaning and feature engineering a dataset exactly once in a single function, then reusing that same cleaned DataFrame for every downstream statistic, SQL query, and visualization, instead of re-cleaning the raw data separately in each analysis step.

Real-world example

Real-world example: A junior analyst asked to "explore this new dataset and report back" is being asked to do exactly what Day 13 practiced: load and clean it once, compute the statistics that matter, cross-check with a couple of SQL-style aggregations, build a small dashboard of plots, and write up the findings in plain English — the exact deliverable an interviewer or manager expects from a first-week data science task.

Implementation

```
        def load_and_clean():
            df = pd.read_csv("titanic.csv")
            df["Age"] = df["Age"].fillna(df["Age"].median())
            df = df.drop(columns=["Cabin"])
            df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
            return df

        def run_statistical_summary(df):
            findings = []
            t_stat, p = stats.ttest_ind(
                df[df.Sex=="male"].Age, df[df.Sex=="female"].Age, equal_var=False
            )
            finding = f"Age difference by sex: p={p:.4f}"
            findings.append(finding)
            return findings

        def build_dashboard(df, corr, out_path="dashboard.png"):
            fig, axes = plt.subplots(2, 3, figsize=(15, 9))
            axes[0, 0].bar(["Died", "Survived"], df["Survived"].value_counts())
            plt.savefig(out_path)
```

- load_and_clean() runs exactly once in main() and its returned DataFrame is passed into every other function — none of the statistics, SQL, or dashboard functions re-read or re-clean the raw CSV themselves
- `findings.append(finding)` — every statistical result is converted into a plain-English string and collected into one list, which write_readme() later turns directly into a README.md without needing to recompute anything
- `subplots(2, 3)` creates a 2-row, 3-column grid of axes; `axes[0, 0]` addresses the top-left plot by (row, column) — this indexing scheme is what makes a multi-panel dashboard possible in one figure

Day 14 — Week 2 review

Definition

Definition — Review day (no new code): a deliberate pause to consolidate Days 8-13 through self-check questions and triage — identifying specifically which concepts need re-study rather than

re-doing every script from scratch.

Real-world example

Real-world example: Before a technical interview, nobody has time to re-solve every practice problem they've ever done. Triage — "I'm solid on joins, shaky on window functions, re-read just that section" — is the actual skill review days are meant to build, and the same skill that makes interview prep efficient instead of overwhelming.

Why this day matters

Days 8-13 moved from Python fundamentals into SQL and a full analysis project quickly. Day 14 exists to catch gaps before Week 3 builds machine learning on top of this foundation — every model from Day 15 onward assumes the cleaning, statistics, and SQL cross-checking habits from this week are already second nature.

Interview prep quiz — technical

Questions about SQL syntax and EDA implementation details.

Q: What's the difference between WHERE and HAVING, and why can't you use an aggregate function like AVG() inside WHERE?

A: WHERE filters individual rows before any grouping occurs, so aggregate values like AVG() don't exist yet at that point in query execution. HAVING filters after GROUP BY has already produced aggregated groups, which is why aggregate functions are only valid there.

Q: What's the difference between INNER JOIN and LEFT JOIN?

A: INNER JOIN only returns rows that have a match in both tables. LEFT JOIN returns every row from the left table regardless of a match, filling in NULL for columns from the right table when no match exists.

Q: What does a CTE (WITH ... AS) actually do, and why use it instead of a nested subquery?

A: It defines a named, temporary result set that can be referenced later in the query, exactly like a temporary table. It doesn't do anything a nested subquery couldn't do, but it makes multi-step logic dramatically more readable, especially when the same sub-result is needed more than once.

Q: What's the difference between RANK() and ROW_NUMBER() when there are ties in the data?

A: RANK() gives tied rows the same rank and then skips the next rank number (1, 2, 2, 4). ROW_NUMBER() always assigns a unique, strictly increasing number regardless of ties (1, 2, 3, 4), even to rows with identical values.

Q: What does PARTITION BY do inside a window function, and how is it different from GROUP BY?

A: PARTITION BY resets the window function's calculation separately within each group, similar to GROUP BY, but it doesn't collapse the rows into one row per group — every original row stays visible with the computed value attached as a new column.

Q: Why does a clean-once EDA pipeline pass the same cleaned DataFrame into every downstream function instead of having each function reload and re-clean the raw CSV?

A: Cleaning in one place guarantees every statistic, SQL query, and plot is working from identical data. If each function re-cleaned independently, a fix applied in one place could be missed in another, silently producing inconsistent results across the same report.

Interview prep quiz — theoretical / conceptual

Questions about why SQL and EDA practices matter in real analysis work.

Q: Why might an interviewer ask you to solve the same problem in both SQL and pandas?

A: They're testing whether you understand the underlying data operation (filtering, grouping, joining) rather than memorized syntax in one tool. In practice, data science roles often require moving fluidly between a database layer (SQL) and an analysis layer (pandas), so comfort in both is a genuine job requirement, not just an interview quirk.

Q: Why is 'explore the data before modeling it' considered a non-negotiable step rather than an optional nice-to-have?

A: Modeling on unexamined data risks building on top of data quality issues — missing values, skewed distributions, mislabeled categories — that silently bias results. EDA surfaces those issues while they're still cheap to fix, before a model has been trained on flawed assumptions.

Q: Why is a plain-English written summary (like the auto-generated README in Day 13) considered part of the actual deliverable, not just a nice extra?

A: Most stakeholders reading an analysis aren't going to read the code or the raw stats output — they need the finding translated into a sentence they can act on. An analysis that never gets communicated in plain language effectively never gets used, no matter how correct the underlying statistics are.

Q: What's the value of a review day with no new code, especially right before Week 3 introduces machine learning?

A: Machine learning in Week 3 assumes the cleaning, SQL, and statistical habits from Week 2 are already automatic, so attention can go toward new modeling concepts instead of re-deriving how to handle missing data. A review day catches shaky foundations while they're still cheap to fix, rather than compounding confusion once new material stacks on top.